

Unit 2

Relational Data Model

Dr. Goonjan Jain

Department of Applied Mathematics

DTU

Introduction

- The Relational Data Model is a conceptual framework used in database management systems (DBMS) to represent and organize data.
- It was proposed by Edgar F. Codd in 1970
- Foundation for modern relational database systems.

Key concepts of Relational Data Model

- Tables (Relations)
- Attributes (Fields)
- Tuples (Rows)
- Domains
- Keys
- Integrity constraints
- Relational operations
- SQL

These concepts collectively define the structure and organization of data within the relational data model, offering a flexible and powerful framework for managing and querying databases.

Tables (Relations)

- A table, also known as a relation, represents a set of related data organized in rows and columns.
- Each table has a unique name and consists of attributes (columns) and tuples (rows).
- The order of tuples is not significant.

Attributes (Fields)

- Attributes represent the columns or fields in a table.
- Each attribute has a distinct name and holds data of a specific type, such as integers, strings, or dates.

Tuples (Rows)

- Tuples are the rows of a table, each representing a single record.
- A tuple consists of a set of values, one for each attribute, and is sometimes referred to as a record.

Domains

- A domain is the set of all possible values that an attribute can take.
- Each attribute is associated with a specific domain, defining the valid range of values it can hold.

Keys

- Keys are attributes or sets of attributes that uniquely identify each tuple in a table.
- A Primary Key is a unique identifier for a tuple within a table.
- A Foreign Key is an attribute that refers to the primary key of another table, establishing relationships between table

Superkey

- a set of one or more attributes that, taken collectively, allow us to uniquely identify an entity in the entity set.
- For example, the *ID* attribute of the entity set *instructor* is sufficient to distinguish one instructor from another. Thus, *ID* is a superkey.
- The *name* attribute of *instructor*, on the other hand, is not a superkey because several instructors might have the same name.
- A superkey may contain extraneous attributes.
- A combination of *ID* and *name* is a superkey for the entity set *instructor*.
- If *K* is a superkey, then so is any superset of *K*.

Candidate Keys

- We are often interested in superkeys for which no proper subset is a superkey.
- Such minimal superkeys are called candidate keys.
- several distinct sets of attributes could serve as a candidate key.
- combination of *name* and *dept name* is sufficient to distinguish among members of the *instructor*.
- Both $\{ID\}$ and $\{name, dept\ name\}$ are candidate keys.
- Although the attributes *ID* and *name* together can distinguish *instructor* tuples, their combination, $\{ID, name\}$, does not form a candidate key, since the attribute *ID* alone is a candidate key.

Primary Key

- denote a candidate key chosen by the database designer as the principal means of identifying entities within an entity set.
- E.g. {ID}
- The remaining candidate keys are called **alternate keys**. E.g. {name, dept name}

Relational Integrity constraints

1. Entity Integrity Constraint:

- This constraint ensures that each table has a primary key uniquely identifying each row or tuple.
- No primary key value can be NULL because a primary key must be able to uniquely identify every row in the table. If a primary key had a NULL value, uniquely identifying that row would be impossible.
- While primary key fields cannot be NULL, other columns in the table can contain NULL values, unless specified otherwise by other constraints

2. Referential Integrity Constraint:

- This constraint is applied between two tables to maintain consistency between related data.
- If a Foreign key in Table 1 references the primary key of Table 2, then every value of the foreign key in Table 1 must either be NULL or match a value in the primary key of Table 2. This ensures that relationships between tables remain consistent.
- This constraint helps maintain the logical relationships between tables, such as enforcing valid references and cascading updates or deletions if defined.

3. Key Constraint (Uniqueness Constraint):

- This constraint ensures that values in certain columns (or combinations of columns) are unique across all rows in a table.
- **Primary Key:** A primary key is a special case of a uniqueness constraint. It uniquely identifies each row and cannot have NULL values. Each table can have only one primary key, composed of one or more columns (attributes).
- **Candidate Keys:** A table can have multiple candidate keys, each of which can potentially serve as a primary key. Out of these, one is selected as the primary key. Candidate keys must also be unique and not contain NULL values.
- **Uniqueness Constraint:** Besides the primary key, other columns or sets of columns can have uniqueness constraints to ensure that no duplicate values exist in those columns.

Primary Key Constraint

- Ensures that each row in a table is uniquely identified by a specific attribute or a set of attributes.
- No two rows can have the same values for the primary key attribute(s).
- Primary keys are used to establish relationships between tables.

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50)  
);
```

Foreign Key Constraint

- Establishes a link between two tables by referencing the primary key of one table as a foreign key in another.
- Ensures referential integrity by preventing the creation of orphaned records.

```
CREATE TABLE Courses (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50)  
);  
  
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)  
);
```

Unique Constraint

- Ensures that all values in a specific column (or combination of columns) are unique.
- Similar to a primary key constraint but does not necessarily enforce the uniqueness across all rows.

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    SSN VARCHAR(9) UNIQUE,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50)  
);
```

Check constraint

- Enforces a condition on the values allowed in a column.
- Restricts the range or format of data that can be stored in a column.

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    OrderDate DATE,  
    TotalAmount DECIMAL(10, 2),  
    CHECK (TotalAmount >= 0)  
);
```

Not-null constraint

- Ensures that a column cannot have a NULL value.
- Requires that every row must have a value for the specified column.

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    Email VARCHAR(100)  
);
```

Referential Integrity

- Referential integrity is a crucial aspect of the relational data model that ensures the consistency and accuracy of relationships between tables in a database.
- It is maintained through the use of foreign key constraints, which link tables based on common attributes.
 - Foreign Key constraint
 - Maintaining referential integrity
 - Cascade actions

Foreign Key Constraint

- Foreign key is a column or a set of columns that references the primary key or a unique key in another table.
- The foreign key establishes a relationship between the two tables, indicating that values in the foreign key column(s) must correspond to values in the referenced primary key column(s) in the related table.
- Foreign keys are used to create relationships

Foreign Key Constraint

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50)  
);  
  
CREATE TABLE Courses (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50)  
);  
  
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)  
);
```

Maintaining Referential Integrity

- Referential integrity ensures that relationships between tables remain valid over time.
- It prevents the creation of "orphaned" records by enforcing that a foreign key value in one table must match a primary key value in another table.

Cascade Actions

- Cascade actions specify what should happen when a referenced primary key is updated or deleted.
- Common options include:
 - **CASCADE:** Updates or deletes the corresponding foreign key values in the referencing table.
 - **SET NULL:** Sets the foreign key values in the referencing table to NULL.
 - **SET DEFAULT:** Sets the foreign key values in the referencing table to their default values.
 - **NO ACTION:** Rejects the update or delete operation if it would violate referential integrity.

Cascade Actions

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50)  
);  
  
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    EmployeeID INT,  
    OrderDate DATE,  
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID) ON DELETE SET NULL  
);
```

Referential Integrity

- Referential integrity ensures that the relationships between tables are maintained, preventing inconsistencies and errors in the database.
- It is a fundamental concept in relational database management systems (RDBMS) that helps maintain the overall quality and reliability of the data.

Assertions

- Assertions in the relational data model are statements that define conditions or rules that must be true for the database to be considered consistent.
- Unlike integrity constraints that are associated with individual tables, assertions can involve multiple tables and express more complex conditions that span across the entire database.
- Assertions are typically used to enforce business rules or conditions that cannot be adequately captured by primary key, foreign key, or other constraints at the table level.

```
CREATE ASSERTION assertion_name  
CHECK (condition);
```

Assertions – Conditions

- The condition specified in an assertion can be any valid logical expression involving one or more tables in the database.
- The condition should evaluate to either TRUE or UNKNOWN for the assertion to be considered satisfied.

```
CREATE ASSERTION student_age_limit  
CHECK (SELECT COUNT(*) FROM Students WHERE Age > 18) <= 5;
```

Assertions - Examples

- Ensuring Consistency Across Tables

```
CREATE ASSERTION total_orders_greater_than_zero  
CHECK (SELECT COUNT(*) FROM Orders) > 0;
```

- Enforcing Business Rules

```
CREATE ASSERTION valid_credit_limit  
CHECK (SELECT COUNT(*) FROM Customers WHERE CreditLimit >= 0) = (SELECT COUNT(*) FROM Customers);
```

Assertions - Activation

- Assertions are typically active by default, meaning the DBMS continuously checks their validity.
- If the condition specified in an assertion becomes false, the DBMS may prevent certain operations that could violate the assertion from occurring (e.g., inserting a new record).

Assertions - Deactivation

- Some DBMS may provide options to temporarily deactivate or disable assertions. This can be useful during data maintenance or specific operations where the assertion condition might be violated temporarily but is expected to be restored.

Assertions

- It's worth noting that while assertions are a part of the relational model, not all relational database management systems (RDBMS) support them directly. Some systems may provide alternatives or additional features to achieve similar outcomes, while others may rely more on triggers, stored procedures, or application logic to enforce complex conditions.

Triggers

- Special types of stored procedures or sets of instructions that are automatically executed ("triggered") in response to specific events or conditions that occur in the database.
- Events can include changes to data (insert, update, delete operations) or changes to the database schema.
- Used to enforce complex business rules, maintain data integrity, and automate certain actions within the database.
- Use Case:
 - Enforcing business rules: Triggers can be used to ensure that specific conditions are met before allowing changes to the database.
 - Maintaining referential integrity: Triggers can enforce complex relationships between tables.
 - Logging changes: Triggers can be used to log modifications to the database for auditing purposes.

Triggers –Event Types

- Triggers are associated with specific events, such as: **BEFORE INSERT:** Triggered before a new row is inserted.
 - **AFTER INSERT:** Triggered after a new row is inserted.
 - **BEFORE UPDATE:** Triggered before an existing row is updated.
 - **AFTER UPDATE:** Triggered after an existing row is updated.
 - **BEFORE DELETE:** Triggered before a row is deleted.
 - **AFTER DELETE:** Triggered after a row is deleted.
- Triggers can be classified as either "BEFORE" triggers or "AFTER" triggers, indicating whether they execute before or after the associated event.

Triggers – Triggering Conditions

- Triggers can have associated conditions (e.g., WHEN clauses) that specify when the trigger should be executed based on certain criteria.
- These conditions help control the circumstances under which the trigger fires.

```
CREATE [OR REPLACE] TRIGGER trigger_name
[BEFORE | AFTER] [INSERT | UPDATE | DELETE] ON table_name
[FOR EACH ROW] -- This clause indicates that the trigger is row-level
[WHEN (condition)] -- Optional condition
BEGIN
    -- Trigger actions or code
END;
```

Trigger - Example

- An example of a trigger that automatically updates a timestamp when a row is modified:

```
CREATE TRIGGER update_timestamp  
BEFORE UPDATE ON employees  
FOR EACH ROW  
SET NEW.last_modified = NOW();
```

- It's important to carefully design and test triggers to ensure they meet the intended requirements without negatively affecting the database's performance or introducing unintended side effects.

Relational Operations

- Relational algebra operations (selection, projection, join, etc.) are used to query and manipulate data in relational databases.